LEGO environment Unity script 1 Share

I'm working on a university 3D Interactive Media project using Unity 6 (6000.3.19f1, URP). Our group project is a LEGO-themed zoo.
I need your help creating a Unity Editor C# script that generates a basic, empty LEGO-style environment to use as a starting point for my own 3D modelling and design.
Requirements:

1. Create a large, flat green LEGO-style baseplate with visible studs.
2. Use realistic LEGO proportions, scaled ×100:
   - 1 stud = 0.8 Unity units
   - 1 brick = 0.96 Unity units tall
   - 1 plate = one-third of a brick's height
3. Create basic LEGO brick meshes using cubes and cylindrical studs.
4. Add reusable LEGO materials with slightly glossy plastic appearances.
5. Set up basic directional lighting with soft shadows.
6. Add an overview camera positioned at an angle so I can see the entire environment.
7. Organise the Unity Hierarchy into clearly named GameObjects.
8. Provide an Editor menu option called "Build Empty LEGO Environment" that generates the environment automatically.
9. Keep the code simple, readable and well-commented so I can understand how it works.
10. Explain how to install, run and test the script in Unity.

Important restrictions:
Do NOT generate any of the following:

- Zoo entrance gates, towers or signs.
- Ticket counters or booths.
- Trees, bushes or flowers.
- Pathways or paw-print decorations.
- Fences or queue barriers.
- Security guards or LEGO minifigures.
- Animals or animal enclosures.
- Any completed zoo structures.

I want ONLY the empty LEGO base environment, including the ground, reusable brick-building components, basic lighting and camera.
I will personally design, build and arrange all the zoo entrance structures and decorations afterward.
Please provide the script in manageable sections and explain the purpose of each section so I can understand and verify the code rather than simply copying it.

Show less

Verifying the Build Area and snapping setup ›

I'll write the Editor script as a single `.cs` file, check the Unity APIs it relies on, then go through it section by section.

Edited a file, created a file, and 5 more steps ›

I've written the script and checked that it compiles against stand-in versions of the Unity classes. That check catches typos and type errors, but it isn't a real Unity run, so please test it in the editor using the steps at the end.

It only builds the baseplate, reusable bricks and plates, materials, lighting, a camera and empty folders in the Hierarchy. It creates no zoo content.

Section 1: LEGO dimensions and settings

Every number the script uses is set here. Real LEGO sizes in millimetres are multiplied by 100, with 1 Unity unit treated as 1 metre, so 8 mm becomes 0.8 units.

csharp

```
const float StudPitch    = 0.8f;              // 8.0 mm: stud-to-stud distance
const float BrickHeight  = 0.96f;             // 9.6 mm
const float PlateHeight  = BrickHeight / 3f;  // 3.2 mm = 0.32
const float StudDiameter = 0.48f;             // 4.8 mm
const float StudHeight   = 0.17f;             // 1.7 mm
const float BrickGap     = 0.02f;             // real bricks are 0.1 mm smaller

const int BaseplateTileStuds = 32;            // classic 32x32 LEGO baseplate
const int BaseplateTilesX    = 2;             // 2 x 2 tiles = 64 x 64 studs =
const int BaseplateTilesZ    = 2;
const float PlasticSmoothness = 0.65f;        // 0 = matte, 1 = mirror
static readonly bool CreateSamplePalette = true;
```

The file also contains two lists:

- **`LegoColours`:** 13 named LEGO colours, such as Green, Red, Blue, Tan and Light Bluish Grey.
- **`PieceCatalogue`:** the bricks and plates to turn into prefabs, written as `(studsX, studsZ, plates)`. A height of 3 plates is a full brick and 1 plate is a plate. To add a 2×6 brick, add `(6, 2, 3)`.

`BrickGap` copies real LEGO, where a 1×1 brick is 7.8 mm wide rather than 8 mm. This leaves the thin seam you see between two bricks side by side. It also stops their touching faces from flickering, which happens when two faces sit in exactly the same spot.

Section 2: Menu item and main build steps

This is the only public method. It calls the other sections in order.

csharp

```
[MenuItem("Tools/LEGO Zoo/Build Empty LEGO Environment")]
public static void BuildEmptyLegoEnvironment()
{
    GameObject oldRoot = FindRootObject(EnvironmentRootName);
    if (oldRoot != null)
    {
        bool rebuild = EditorUtility.DisplayDialog("Rebuild LEGO Environment?",
        if (!rebuild) return;
    }

    Undo.SetCurrentGroupName("Build Empty LEGO Environment");
    int undoGroup = Undo.GetCurrentGroup();

    if (oldRoot != null) Undo.DestroyObjectImmediate(oldRoot);
    DisableDefaultSceneObjects();

    EnsureFolders();
    Dictionary<string, Material> materials = CreateMaterials();
    List<GameObject> piecePrefabs = CreatePiecePrefabs(materials["Red"]);
    GameObject baseplatePrefab = CreatePiecePrefab(
        $"Baseplate_{BaseplateTileStuds}x{BaseplateTileStuds}",
        BaseplateTileStuds, BaseplateTileStuds, PlateHeight, 0f, materials["Gre

    GameObject root     = new GameObject(EnvironmentRootName);
    GameObject lighting = CreateChild("Lighting", root);
    GameObject cameras  = CreateChild("Cameras", root);
    GameObject ground   = CreateChild("Ground", root);

    BuildGround(ground, baseplatePrefab);
    BuildLighting(lighting);
    BuildOverviewCamera(cameras);
    if (CreateSamplePalette)
        BuildSamplePalette(CreateChild("Brick Palette (samples - safe to delete

    Undo.RegisterCreatedObjectUndo(root, "Create LEGO Environment");
    root.transform.SetSiblingIndex(0);
    EnsureBuildArea();
    SetupLegoSnapping();

    Undo.CollapseUndoOperations(undoGroup);
    AssetDatabase.SaveAssets();
    EditorSceneManager.MarkSceneDirty(SceneManager.GetActiveScene());
    Selection.activeGameObject = root;
}
```

- **One undo step:** the Undo group lets a single Ctrl+Z remove the whole build.
- **Safe to run again:** if you run it a second time, it asks first, then deletes and rebuilds only the `LEGO Environment` object. `Build Area` is a separate top-level object that the script never deletes, so your own work there is safe.

Section 3: Folders and reusable materials

csharp

```
Shader shader = Shader.Find("Universal Render Pipeline/Lit");
if (shader == null) { Debug.LogWarning("..."); shader = Shader.Find("Standard")

foreach (var entry in LegoColours)
{
    string colourName = entry.name;
    Color colour = entry.colour;
    string path = $"{MaterialFolder}/LEGO_{colourName.Replace(" ", "")}.mat";
    Material material = AssetDatabase.LoadAssetAtPath<Material>(path);

    if (material == null)
    {
        material = new Material(shader);
        material.color = colour;                                // URP: _BaseColo
        material.SetFloat("_Smoothness", PlasticSmoothness);  // URP Lit glossi
        material.SetFloat("_Glossiness", PlasticSmoothness);  // Standard-shade
        material.SetFloat("_Metallic", 0f);                    // plastic is not
        material.enableInstancing = true;
        AssetDatabase.CreateAsset(material, path);
    }
    materials[colourName] = material;
}
```

- **Saved as real files:** materials go into `Assets/LEGO/Materials`, so you can drag them onto anything.
- **The glossy look:** it comes from Smoothness 0.65 with Metallic at 0.
- **Your edits are kept:** if a material already exists, the script reuses it instead of overwriting it, so changes you make in the Inspector survive a rebuild.

Section 4: Brick mesh generation

All pieces, including the baseplate, are built by one method. It takes Unity's built-in cube and cylinder meshes, places and scales copies of them, and merges them into a single mesh.

csharp

```
static Mesh BuildPieceMesh(string meshName, int studsX, int studsZ, float bodyH
{
    Mesh cube     = GetPrimitiveMesh(PrimitiveType.Cube);      // 1x1x1, centre
    Mesh cylinder = GetPrimitiveMesh(PrimitiveType.Cylinder);  // 1 wide, 2 tal
    float width = studsX * StudPitch;
    float depth = studsZ * StudPitch;
    var parts = new List<CombineInstance>();

    // Body
    parts.Add(new CombineInstance {
        mesh = cube,
        transform = Matrix4x4.TRS(new Vector3(width / 2f, bodyHeight / 2f, dept
                                  Quaternion.identity,
                                  new Vector3(width - gap, bodyHeight, depth -

    // Studs
    Vector3 studScale = new Vector3(StudDiameter, StudHeight / 2f, StudDiameter
    for (int x = 0; x < studsX; x++)
        for (int z = 0; z < studsZ; z++)
        {
            Vector3 studCentre = new Vector3((x + 0.5f) * StudPitch, bodyHeight
            parts.Add(new CombineInstance { mesh = cylinder,
                transform = Matrix4x4.TRS(studCentre, Quaternion.identity, stud
        }

    Mesh mesh = new Mesh { name = meshName, indexFormat = IndexFormat.UInt32 };
    mesh.CombineMeshes(parts.ToArray(), true, true);
    return SaveMeshAsset(mesh, $"{MeshFolder}/{meshName}.asset");
}
```

- **Pivot at the corner:** the pivot point is the bottom-left-front corner of each piece. Because of this, any piece of any size lines up with the studs when you move it in 0.8 steps.
- **Studs:** Unity's cylinder is 2 units tall, so its Y scale is half the stud height.
- `UInt32`**:** a 32×32 baseplate has more than 65,535 vertices, which is above Unity's default mesh limit, so the script switches to 32-bit indices.
- **Helpers:**
  - `GetPrimitiveMesh` borrows the built-in mesh from a temporary primitive object.
  - `SaveMeshAsset` updates an existing mesh file in place, so prefabs that use it stay linked.

Section 5: Reusable prefabs (the brick library)

csharp

```
static GameObject CreatePiecePrefab(string pieceName, int studsX, int studsZ,
                                    float bodyHeight, float gap, Material mater
{
    Mesh mesh = BuildPieceMesh(pieceName, studsX, studsZ, bodyHeight, gap);
    GameObject temp = new GameObject(pieceName);
    temp.AddComponent<MeshFilter>().sharedMesh = mesh;
    temp.AddComponent<MeshRenderer>().sharedMaterial = material;

    BoxCollider box = temp.AddComponent<BoxCollider>();
    box.center = new Vector3(studsX * StudPitch / 2f, bodyHeight / 2f, studsZ *
    box.size   = new Vector3(studsX * StudPitch - gap, bodyHeight, studsZ * Stu

    GameObject prefab = PrefabUtility.SaveAsPrefabAsset(temp, $"{PrefabFolder}/
    Object.DestroyImmediate(temp);
    return prefab;
}
```

`CreatePiecePrefabs` runs this for every catalogue entry, which gives you `Brick_1x1` to `Brick_2x4` and `Plate_1x1` to `Plate_4x4` in `Assets/LEGO/Prefabs`.

Each piece is one object with one mesh, so dropping a material on it recolours the whole brick. The collider covers the body only, not the studs. One thing to know: rebuilding overwrites these prefab files, so don't customise them directly. Make your own prefab variants instead.

Section 6: Ground (green baseplate)

csharp

```
float tileSize = BaseplateTileStuds * StudPitch;   // 25.6
float startX = -BaseplateTilesX * tileSize / 2f;   // centre the ground on (0,0
float startZ = -BaseplateTilesZ * tileSize / 2f;

for (int tx = 0; tx < BaseplateTilesX; tx++)
    for (int tz = 0; tz < BaseplateTilesZ; tz++)
    {
        var tile = (GameObject)PrefabUtility.InstantiatePrefab(baseplatePrefab,
        tile.name = $"Baseplate Tile ({tx},{tz})";
        tile.transform.localPosition = new Vector3(startX + tx * tileSize, -Pla
    }
```

The ground is laid out from 32×32 baseplate tiles that all share one mesh, which keeps file sizes small. The tiles sit 0.32 units below zero, so **the top of the baseplate is exactly Y = 0**. That gives you an easy rule for stacking:

- First layer: Y = 0
- Each brick on top: +0.96
- Each plate on top: +0.32

If you need more space, increase `BaseplateTilesX` and `BaseplateTilesZ`, or duplicate a tile.

Section 7: Lighting

csharp

```
sunObject.transform.rotation = Quaternion.Euler(50f, -35f, 0f);
Light sun = sunObject.AddComponent<Light>();
sun.type = LightType.Directional;
sun.color = new Color(1f, 0.96f, 0.90f);
sun.intensity = 1.3f;
sun.shadows = LightShadows.Soft;
sun.shadowStrength = 0.75f;
RenderSettings.sun = sun;

RenderSettings.ambientMode = AmbientMode.Trilight;
RenderSettings.ambientSkyColor     = new Color(0.62f, 0.70f, 0.82f);
RenderSettings.ambientEquatorColor = new Color(0.50f, 0.52f, 0.55f);
RenderSettings.ambientGroundColor  = new Color(0.30f, 0.30f, 0.28f);
```

There is one slightly warm “sun” light, angled to cast soft shadows. Gradient ambient light fills in the shaded sides so they don’t go black.

Section 8: Overview camera

csharp

```
Camera cam = cameraObject.AddComponent<Camera>();
cam.fieldOfView = 60f; cam.nearClipPlane = 0.1f; cam.farClipPlane = 500f;
cameraObject.AddComponent<AudioListener>();

float groundSize = Mathf.Max(BaseplateTilesX, BaseplateTilesZ) * BaseplateTileS
cameraObject.transform.position = new Vector3(0f, groundSize * 0.7f, -groundSiz
cameraObject.transform.LookAt(Vector3.zero);

if (SceneView.lastActiveSceneView != null)
    SceneView.lastActiveSceneView.AlignViewToObject(cameraObject.transform);
```

The camera's position is worked out from the ground size, so it still frames everything if you make the baseplate bigger. It looks down at about 38°. The Scene view is also moved to the same viewpoint.

Section 9: Sample palette, snapping and helpers

- `BuildSamplePalette`: places one of each piece in different colours along the front-left edge of the baseplate, lined up on the studs. It's only there to check proportions, and you can delete it or set `CreateSamplePalette = false`.
- `SetupLegoSnapping`:

```csharp
EditorSnapSettings.move = new Vector3(StudPitch, PlateHeight, StudPitch);
EditorSnapSettings.rotate = 90f;
```

When you hold **Ctrl** (Cmd on Mac) while dragging with the Move tool, bricks jump one stud sideways or one plate up and down. Rotation snaps in 90° steps.

- `DisableDefaultSceneObjects`: turns off the default `Main Camera` and `Directional Light` without deleting them, so you don't have two cameras and two suns.
- `FindRootObject` and `CreateChild`: small helpers for finding and creating objects in the Hierarchy.

Resulting Hierarchy

```
LEGO Environment              ← generated (rebuilt on each run)
├── Lighting
│   └── Sun (Directional Light)
├── Cameras
│   └── Overview Camera
├── Ground
│   └── Baseplate Tile (0,0) … (1,1)
└── Brick Palette (samples - safe to delete)
Build Area                    ← empty, yours, never touched
```

How to install, run and test

1. **Install:** in the Project window, create the folder `Assets/Editor` and put `LegoEnvironmentBuilder.cs` inside it. The folder must be called `Editor`, or Unity will refuse to build the game because of the `UnityEditor` code. Wait for Unity to compile, and check the Console has no red errors.
2. **Run:** open a scene (File → New Scene → **Basic (URP)** works well), then click **Tools → LEGO Zoo → Build Empty LEGO Environment**. Save the scene afterwards.
3. **Test:**
   - **Proportions:** select `Brick_1x1` from the palette. Its Box Collider should be 0.78 × 0.96 × 0.78. A `Plate_1x1` should be exactly a third as tall.

- **Grid:** drag `Assets/LEGO/Prefabs/Brick_2x4` into `Build Area`, set its Y to 0, then hold Ctrl and drag it. Its studs should line up with the baseplate studs. Stack a second brick at Y = 0.96 and it should sit flush.
- **Materials:** drag `LEGO_Blue` onto a brick. The whole brick should turn blue and look glossy.
- **Camera:** press Play. The Game view should show the whole baseplate at an angle, with soft shadows.
- **Undo and rebuild:** Ctrl+Z should remove everything at once. Running the menu again should ask before rebuilding.

If shadows fade out on the far side of the baseplate: this is caused by a setting in the URP asset, not the script. URP's Max Shadow Distance usually defaults to 50, which is a little short for a 51-unit plate. Select your URP Asset (under Project Settings → Graphics / Quality), raise **Shadows → Max Distance** to about 120, and make sure **Soft Shadows** is ticked.

Shared LegoEnvironmentBuilder.cs

I've sent the full script as `LegoEnvironmentBuilder.cs`. It's the same code as the sections above, but with all the comments and the full colour and brick lists.

The first time you run it in Unity, please check the Console for warnings. In particular, a "URP Lit shader not found" warning means URP isn't the active render pipeline, and the materials will fall back to the Standard shader.

Can you explain the Unity Editor script step by step in simple terms?

Think of the script as a robot that does, in a few seconds, what you would otherwise click through by hand: it creates folders and materials, builds brick shapes, places a baseplate, adds a light and a camera, and tidies up the Hierarchy. Here's what happens when you click the menu option, in order.

1. It adds a button to Unity's menu

The line `[MenuItem("Tools/LEGO Zoo/Build Empty LEGO Environment")]` tells Unity to put a new option in the Tools menu. Clicking it runs the `BuildEmptyLegoEnvironment` method, which acts like a to-do list and calls the other methods in order. That's also why the file must sit in an `Editor` folder: the script controls the Unity editor itself, so it isn't part of your finished game.

2. It reads the "rulebook" of measurements

The constants at the top are the real LEGO sizes, scaled up 100 times:

- One stud spacing is **0.8** units. That's the width of a 1×1 brick.
- One brick is **0.96** units tall.
- One plate is a third of that, **0.32** units.
- A stud is 0.48 wide and 0.17 tall.

Everything else in the script works from these numbers, so if they're right, every piece comes out in proportion. Changing one number here changes it everywhere.

3. It checks whether you've run it before

If a `LEGO Environment` object already exists, a pop-up asks whether to rebuild. If you say yes, it deletes only that object. Your `Build Area` is kept safe. It also records everything as one undo step, so a single Ctrl+Z reverses the whole build.

4. It hides Unity's default camera and light

A new scene comes with a `Main Camera` and a `Directional Light`. The script switches them off without deleting them, so you don't end up with two cameras and two suns fighting each other.

5. It creates folders in your Project window

It makes `Assets/LEGO/Materials`, `Meshes` and `Prefabs`. If they already exist, it skips this step.

6. It makes the LEGO colours (materials)

For each colour in the list (green, red, blue, yellow and so on), it creates a material file using URP's "Lit" shader. Two settings give the plastic look:

- **Smoothness 0.65:** shiny, but not a mirror.
- **Metallic 0:** plastic isn't metal.

If a material already exists, it leaves it alone, so your own tweaks aren't lost.

7. It builds the brick shapes (meshes)

This is the clever part. A LEGO brick is just a box with little cylinders on top, so for each brick the script:

1. Takes Unity's built-in **cube** and stretches it into the brick's body. For a 2×4 that's 3.2 long, 1.6 wide and 0.96 tall, minus a tiny gap.
2. Takes Unity's built-in **cylinder**, shrinks it into a stud, and copies it onto every stud position on top.
3. **Merges** all those pieces into one single shape and saves it as a file.

The tiny gap (0.02) copies real LEGO, where bricks are slightly smaller than their grid so they fit together. It's also why you see a thin line between neighbouring bricks.

The **pivot point is the bottom corner** of each brick. That means when you move bricks in steps of 0.8, they always land neatly on the studs, whatever their size.

8. It turns those shapes into reusable prefabs

Each shape becomes a prefab, a ready-made object you can drag into the scene as many times as you like. Each one has:

- a **MeshFilter**, which holds the shape
- a **MeshRenderer**, which draws it with a material
- a **BoxCollider**, which gives it a solid body you can click and that physics can use

Because each brick is one object, dropping a colour on it recolours the whole brick.

9. It lays the green baseplate

The baseplate is made the same way: it's really just a very large, flat 32×32 "plate". The script places a 2×2 grid of these tiles (64×64 studs in total), centred on the middle of the world. It lowers them by one plate height so that **the top surface is exactly Y = 0**. That gives you a simple rule:

- A brick sitting on the ground goes at **Y = 0**.
- The next brick on top goes at **Y = 0.96**.

10. It adds the sun

It creates a directional light, which works like the sun: one direction, lighting everything equally. The light is:

- tilted to come in from the side, so it casts shadows
- slightly warm in colour
- set to **soft shadows**

It also adds a gentle background light, so the shaded sides of bricks aren't pitch black.

11. It adds the overview camera

It places a camera up high and back from the baseplate, then points it at the centre. The position is worked out from the baseplate size, so if you make the ground bigger, the camera automatically moves back to fit it all in. The Scene view is moved to the same viewpoint too.

12. It places sample bricks (optional)

It puts one of each brick and plate near the front edge in different colours, just so you can check the sizes look right. You can delete them freely.

13. It sets up snapping

It changes Unity's snap settings, so that when you **hold Ctrl and drag**:

- a brick moves one stud sideways, or one plate up or down
- rotation snaps in 90° turns

That makes building feel like real LEGO.

14. It tidies up and finishes

It arranges everything under named folders in the Hierarchy (Lighting, Cameras, Ground, Brick Palette). It also creates an empty `Build Area` for your own zoo work.

Finally, it saves the files, marks the scene as changed and selects the result.

In one sentence: the script uses real LEGO measurements to build brick shapes from stretched cubes and small cylinders, saves them as reusable prefabs with glossy colours, lays a green studded baseplate whose top sits at Y = 0, and adds a sun, a camera and grid snapping so you can start building straight away.

If any step is still unclear, tell me which one, and I can walk through that part of the code line by line.

> also I was thinking why we using an Editor script instead of manually creating every LEGO brick in Unity?

Mostly because of how many studs there are, and how exact they need to be. Here are the main reasons.

1. There are far too many studs to place by hand

The baseplate alone is 64 × 64 = **4,096 studs**. Placing each cylinder by hand means creating it, typing its position and setting its scale, thousands of times. The script does the same job with a short loop in about a second. The 14 bricks and plates add another 50 or so studs on top of that.

2. Every measurement is exact

By hand it's easy to type 0.79 instead of 0.8, or nudge a stud slightly off-centre. Small errors add up: after 30 studs a brick can be visibly out of line with the baseplate. The script calculates every position from the same rule, `(x + 0.5) × 0.8`, so every stud on every brick lines up perfectly.

3. Changes are easy

Say your lecturer or group decides the studs look too small, or the baseplate needs to be bigger. Done manually, you'd have to rebuild everything. With the script, you change **one number** at the top and click the menu again. Every brick, the baseplate and even the camera position update automatically.

4. Everyone in your group gets the same result

In a group project, if each person makes their own bricks, you end up with slightly different sizes, colours and pivot points, and pieces from different people won't fit together. With the script, everyone runs the same menu option and gets identical bricks, materials and scale. It's a shared standard.

5. You can undo and rebuild safely

If someone accidentally deletes or messes up the baseplate, you just run the menu again. Nothing is lost, because the "instructions" for building it live in the script, not in someone's memory.

6. It runs faster in the game

When you build by hand, each stud is usually its own object, so the baseplate becomes thousands of objects in your Hierarchy. The script merges each brick into **one mesh**, so a whole 32×32 baseplate tile is one object. That keeps the Hierarchy clean, makes the editor run more smoothly, and means Unity has less work to do when drawing each frame.

7. It's a good fit for this course

Generating geometry from code, using prefabs and keeping projects organised are all real skills in interactive media. Being able to explain *why* you automated the repetitive parts can strengthen your project report.

When doing it by hand is better

The script isn't the right tool for everything:

- **Unique or creative pieces**, such as your entrance design, sloped bricks, arches and decorations, are better modelled by hand in Unity or in Blender, where you can shape them artistically.
- **Arranging the scene** is also best done by hand. Deciding where things go is your design work, and a script can't decide what looks good.

The script handles the boring, repetitive and maths-heavy work (the ground, the basic bricks, the setup), so you can spend your time on the creative part: designing and building the zoo yourself.

I want to manually add my own structures to the environment how to do tht

Planning the layout of the build steps and models ›

You build on top of what the script made, using the brick prefabs, your own models, or both. The one rule to remember: **always put your work inside** `Build Area`, because the script never deletes that object. Anything you put inside `LEGO Environment` gets deleted when you rebuild.

1. Place your first brick

1. In the Project window, open `Assets/LEGO/Prefabs`.
2. Drag a prefab, for example `Brick_2x4`, onto `Build Area` in the Hierarchy. That makes it a child of Build Area.
3. In the Inspector, set its **Position Y to 0** so it sits on the baseplate.

2. Move bricks so they line up with the studs

- Select the brick, press **W** (Move tool), then **hold Ctrl** (Cmd on Mac) while dragging an arrow. It jumps exactly one stud sideways, or one plate up or down.
- Or type positions directly in the Inspector:
  - **X and Z:** any multiple of 0.8 (such as 0, 0.8, 1.6, −4.0) lines up with the studs.
  - **Y:** use the stacking heights below.

Layer	Y position
On the baseplate	0
On top of 1 plate	0.32
On top of 1 brick	0.96
On top of 2 bricks	1.92
On top of 3 bricks	2.88

Every brick is 0.96 tall and every plate is 0.32, so just keep adding.

3. Rotate, copy and recolour

- **Rotate:** press **E**, then hold Ctrl while rotating to turn in 90° steps. Rotated bricks still line up because the pivot is on a stud corner.
- **Duplicate:** press **Ctrl+D**, then Ctrl-drag the copy into place. This is the fastest way to build walls.
- **Recolour:** drag a material from `Assets/LEGO/Materials` (for example `LEGO_Yellow`) onto the brick in the Scene view.
- **Precise alignment:** hold **V** while moving to snap a brick's corner exactly onto another brick's corner.

4. Organise your structures

Inside `Build Area`, right-click → **Create Empty** and give it a name for each structure, such as `Entrance Gate` or `Ticket Booth`. Then drag that structure's bricks into it. Moving the parent moves the whole structure at once.

```
Build Area
├── Entrance Gate
|   ├── Left Pillar
|   └── Right Pillar
└── Ticket Booth
```

5. Save finished structures as your own prefabs

When a structure is finished, drag its parent object from the Hierarchy into a new folder, such as `Assets/Zoo/Prefabs`. Now you can reuse it, and your group members can place it too.

Don't edit the prefabs in `Assets/LEGO/Prefabs` directly. Running the script again overwrites them. If you want a changed version, right-click the prefab → **Create → Prefab Variant** and edit the variant instead.

6. Adding your own 3D models (for example from Blender)

For pieces the basic bricks can't make, like slopes, arches, signs or animals:

1. **Model at the right scale:** 1 Unity unit = 1 metre in Blender, so a 1×1 brick is 0.8 m wide and 0.96 m tall. A good habit is to drag a `Brick_1x1` next to your imported model and compare sizes.
2. **Put the pivot (origin) at the bottom corner** of the model. Then it snaps to the stud grid like the bricks do.
3. **Export as FBX**, with **Apply Transform** ticked in Blender's export settings, and drop the file into `Assets/Zoo/Models`.
4. **Check the scale:** select the FBX in Unity and check its import settings. If it comes in 100 times too big or too small, adjust **Scale Factor** and click Apply.
5. **Use the LEGO materials:** drag the FBX into `Build Area`, then drag LEGO materials onto it so it matches the bricks.

7. Save often

Press **Ctrl+S** to save the scene regularly. If your group uses Git or Unity Version Control, commit your work and your `Assets/Zoo` folder. You don't need to commit `Assets/LEGO`, since anyone can regenerate it by running the menu option.

A good workflow: sketch the structure on paper or squared paper (one square = one stud), then build the base layer at Y = 0, stack upwards using Ctrl-drag and Ctrl+D, group it under a named parent, and save it as a prefab.

